UNIVERSITY OF THE WITWATERSRAND, JOHANNESBURG
School of Computer Science & Applied Mathematics

Advanced Analysis of Algorithms
Lab 1: Uninformed Search

1 Introduction

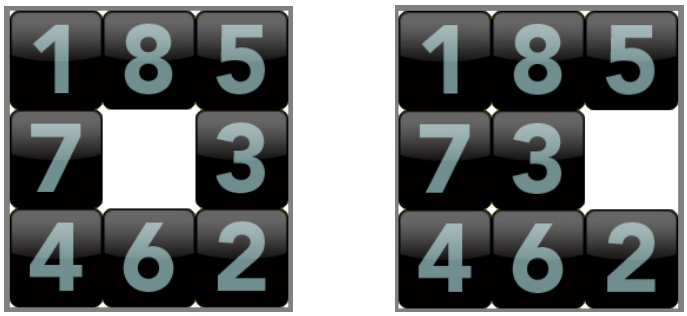
In this lab, we'll be implementing breadth-first search on the 8-puzzle problem. Please note that this lab is **not for marks**. While there are submission links on Moodle and some of the submissions have mark indications, this is purely for your own information and to help you align on what to expect for the lab test. While the lab is not compulsory, I highly recommend that you complete it well in advance of the lab test to ensure you are prepared.

2 8-Puzzle

The 8-Puzzle is a game in which a player is required to slide tiles around on a 3×3 grid in order to reach a goal configuration. If you have never played before, I recommend trying it out here: `http://www.artbylogic.com/puzzles/numSlider/numberShuffle.htm?rows=3&cols=3&sqr=1`.

There is one empty space that an adjacent tile can slide into. An alternative (and simpler) view is that we are simply moving the blank square around the grid. Note that not all actions can be used at all times. For example, the blank tile cannot be shifted up if it is already on the top row!

The transition below illustrates the blank tile being moved to the `right`.



(a) (b)

Figure 1: Starting in the configuration (a), the agent executes the action `right`. This moves the blank tile to the right, exchanging its position with 3, as seen in (b).

Submission 1: Implementing moves (10 marks)

The first step is to implement action dynamics for a given game state. Write a Python programme that takes in an initial configuration and an action, and outputs the resulting configuration.

Input

Input consists of two lines. The first line is a single string of digits and the hash symbol. The position of the digit in the string is its location on the board, starting from top-left and moving to bottom right. The hash represents the blank space. For example, the board configuration given in Figure 1(a) would be represented by the string `1857#3462`. The second line is the action taken, which is one of either `UP`, `DOWN`, `LEFT` or `RIGHT`.

Output

The output should be formatted as a single string of digits and the hash symbol. The position of the digits in the string is its location on the board, starting from top-left and moving to bottom right. The hash represents the blank space.

Example Input-Output Pairs

Sample Input #1	Sample Input #2	Sample Input #3
`1857#3462` `LEFT`	`18573#462` `UP`	`78651#432` `DOWN`
Sample Output #1	**Sample Output #2**	**Sample Output #3**
`185#73462`	`18#735462`	`78651243#`

Submission 2: Implementing available actions (10 marks)

The next step is to determine which actions are available in which states. Write a Python programme that takes in an initial configuration, and outputs which actions are available at the given state.

Input

Input consists of a single string of digits and the hash symbol. The position of the digit in the string is its location on the board, starting from top-left and moving to bottom right. The hash represents the blank space. For example, the board configuration given in Figure 1(a) would be represented by the string `1857#3462`.

Output

Output the list of available actions, each on their own line. The actions should be output in the order `UP`, `DOWN`, `LEFT`, `RIGHT`.

Example Input-Output Pairs

Sample Input #1	Sample Input #2	Sample Input #3
`1857#3462`	`18573#462`	`78651432#`
Sample Output #1	**Sample Output #2**	**Sample Output #3**

`UP`
`DOWN`
`LEFT`
`RIGHT`
	`UP`
	`DOWN`
	`LEFT`
		`UP`
		`LEFT`

Visualising the board

While we generally solve the search problem using an abstract representation of the state – it can often be useful to visualise the state in a more naturalistic way. Write a Python script that reads in a single string, stores that information in an appropriate data structure, and then outputs it as a 3×3 square. This would be completely optional and just serves as a hint towards a step which may help you debug the next steps in the submission.

Input

Input consists of a single string of digits and the hash symbol. The position of the digits in the string is its location on the board, starting from top-left and moving to bottom right. The hash represents the blank space. For example, the board configuration given in Figure 1(a) would be represented by the string `1857#3462`.

Output

Print out the board as a 3×3 matrix, where each value is separated by a space.

Example Input-Output Pairs

Sample Input #1

```
1857#3462
```

Sample Output #1

```
1 8 5
7 # 3
4 6 2
```

Sample Input #2

```
18573#462
```

Sample Output #2

```
1 8 5
7 3 #
4 6 2
```

Sample Input #3

```
78651#432
```

Sample Output #3

```
7 8 6
5 1 #
4 3 2
```

Submission 3: Breadth-first search (40 marks)

Now that we've implemented the problem, we can implement our search strategies. First, write a Python programme that accepts an initial and goal state, and returns the cost of the shortest path between the two, given that every action has a cost of 1.

Input

Input consists of two lines. The first line is a representation of the initial state, and the second line is the goal state.

Output

Output the cost of the optimal plan from the start to the goal state.

Example Input-Output Pairs

Sample Input #1	Sample Input #2	Sample Input #3
78651#432 12345678#	1857#3462 185#73462	1857#3462 78651432#
Sample Output #1	**Sample Output #2**	**Sample Output #3**
25	1	20

Submission 4: Experimenting with Our Algorithm

Now that we've implemented the problem and BFS we can experiment with the time complexity of the algorithm. In many cases the domain of a problem is too complicated to do a full analysis of the problem directly (and definitely not from scratch). Thus, we begin by trying to understand the problem and get some intuition from our experiments.

To achieve this we need to be able to control the complexity of the problem we ask the BFS to solve. Write a Python program that generates game boards which require k moves to solve. Begin with the solved game state $12345678\#$ and uniformly sample valid actions to move the blank square around the board until the desired number of moves have been made. A valid action cannot move the blank square off the 3×3 grid and it cannot take the game into an already encountered state. Once the desired number of moves have been made, pass the resulting game state into your BFS function and set the goal state to be $12345678\#$ (the solved puzzle).

Repeat this process for variable solution depths k and plot this relationship with the time and space complexity of your BFS algorithm. Measure time complexity using wall clock time from the beginning of the BFS function to the end. Measure space complexity by the number of nodes which are encountered (expanded on) until the solution is found. For a given value of k repeat the algorithm 5 times and plot both the mean and 2 standard deviations for both complexity measures.

Input

There is no input as you will begin with $12345678\#$ as the game state and choose a value of k to run the experiment.

Things to Consider

The types of things you should take note of and that I may ask (I may also show you a plot like the one I have asked you to create and ask questions from it):

1. What is the time and space complexity of your algorithm that *generates* initial states? Please explain your answer. (5 marks)
2. Why would you *not* randomly place the digits and blank space into an arbitrary configuration when generating initial game states? (5 Marks)
3. Why would you need to ensure that you do not encounter already seen states when *generating* the initial game states. (5 Marks)
4. How large is the variance in time and space complexity of the BFS and why might there be some variance in these complexity measures for a given k? (5 Marks)
5. What is the time and space complexity of the BFS according to your experimental results? How does this compare to what you would predict or expect? (5 marks)